

CS 25-12 Capstone Design Document

Oscar Eriksen, An Ly, Johnathan Maciejewski, Rithikaa Prakash

Version 1.8

10/24/2024



Table Of Contents

Roster	2
Architectural Approach.....	3
Use Cases	4
Logical View	12
Process Diagrams	17
Development Diagrams	22
Physical Diagram.....	24

Roster

Sponsor

Name	Phone	Email
Deborah Boone		

Advisor

Name	Phone	Email
Ritika Kanade	N/A	

Team

Name	Phone	Email
Oscar Eriksen	(425)-979-1773	oeriksen@seattleu.edu
An Ly		
Johnathan Maciejewski		
Rithikaa Prakash		

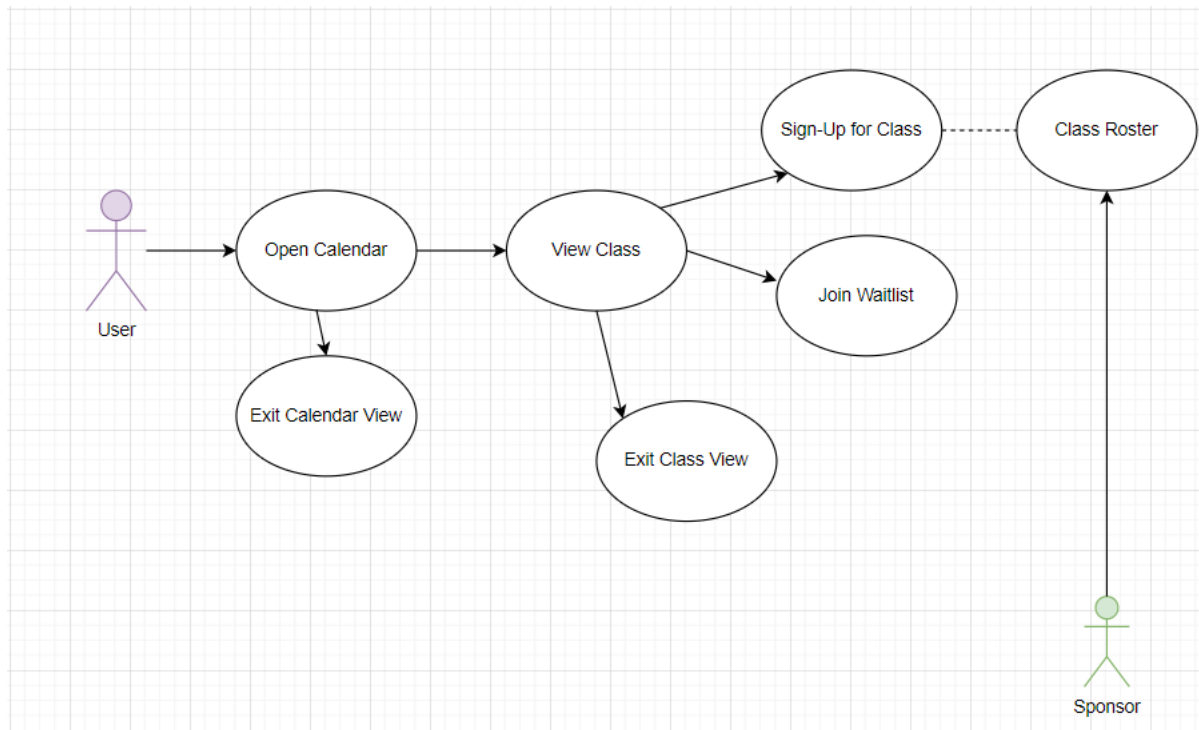
Architectural Approach

For our project, A **Monolithic Architecture** will work well for the app. This pattern integrates all components of the system into a single, cohesive application. The app's core functionalities—fabric inventory management, project storyboarding, calendar scheduling, and video guides—will all operate within one unified codebase. A monolithic architecture is particularly beneficial in the initial phases of the project because it offers simplicity in both deployment and maintenance. By streamlining the development process our team can focus on building features and deploying updates without needing to manage multiple services.

Use Cases

A user logs into the fabric stash app to manage their inventory, plan a project, and schedule deadlines. The user uploads fabric details, organizes them into a storyboard, sets a project timeline using the calendar, and watches video guides for assistance. The app handles these operations within a single, unified system, providing a seamless user experience.

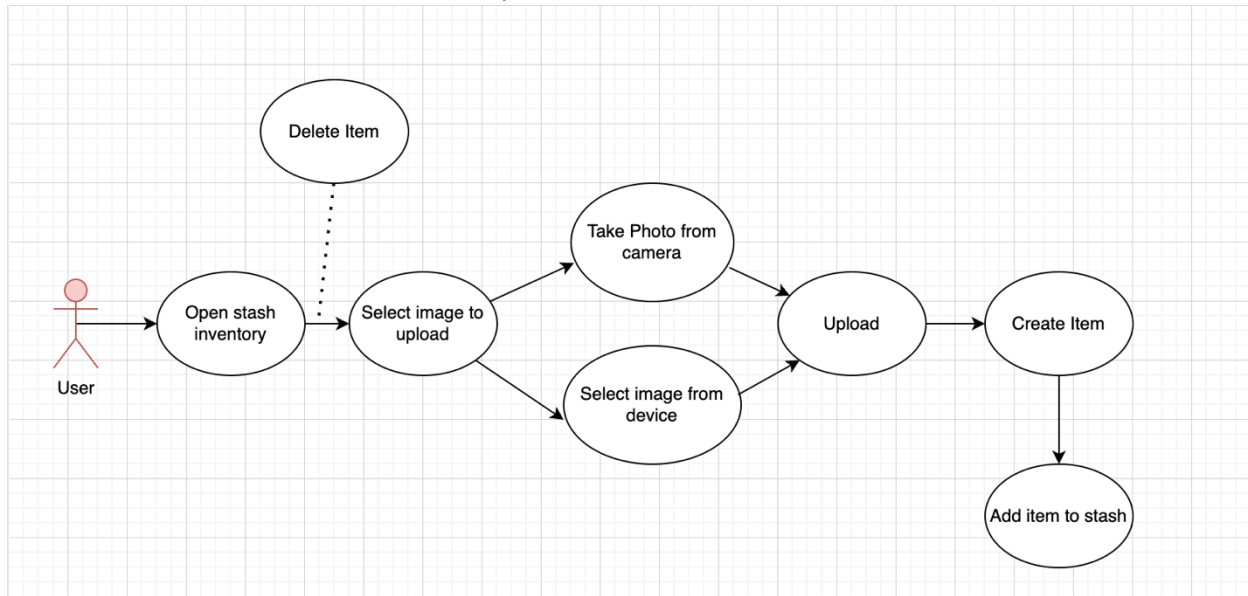
Case 1: User Signs Up for a Class via Calendar



Use Case	User Signs Up for a Class via Calendar
Actors	Primary Actor: User (Purple) Supporting Actor: Sponsor (Green)
Preconditions	The user must be logged into the web app. The user must have access to the calendar page.
Description	The user wants to view available classes through the calendar function and sign up for a class of their choice.
Basic Flow	1. The user logs into the web app. 2. The user selects the "View Calendar" option. 3. The system displays a calendar with available class schedules.

	4. The user selects a desired class from the calendar. 5. The system displays the class details and a "Sign Up" button. 6. The user clicks "Sign Up." 7. The system confirms the class registration and sends a confirmation notification to the user. 8. The system adds the user to the class roster. 9. The sponsor is able to view the class roster.
Alternative Flows	1. The user clicks "Join Waitlist" on the class details page since the class is full. 2. The system confirms they have been added to the waitlist. 3. If space becomes available, the user is added to the class roster.
Postconditions	The user is successfully registered for the class. The class roster is updated to reflect the user's registration.
Exceptions	Network Failure: If the connection is lost during registration, the system notifies the user to try again later. If a double booking occurs at the exact same time for 2 different users, the system notifies both users to try again later.

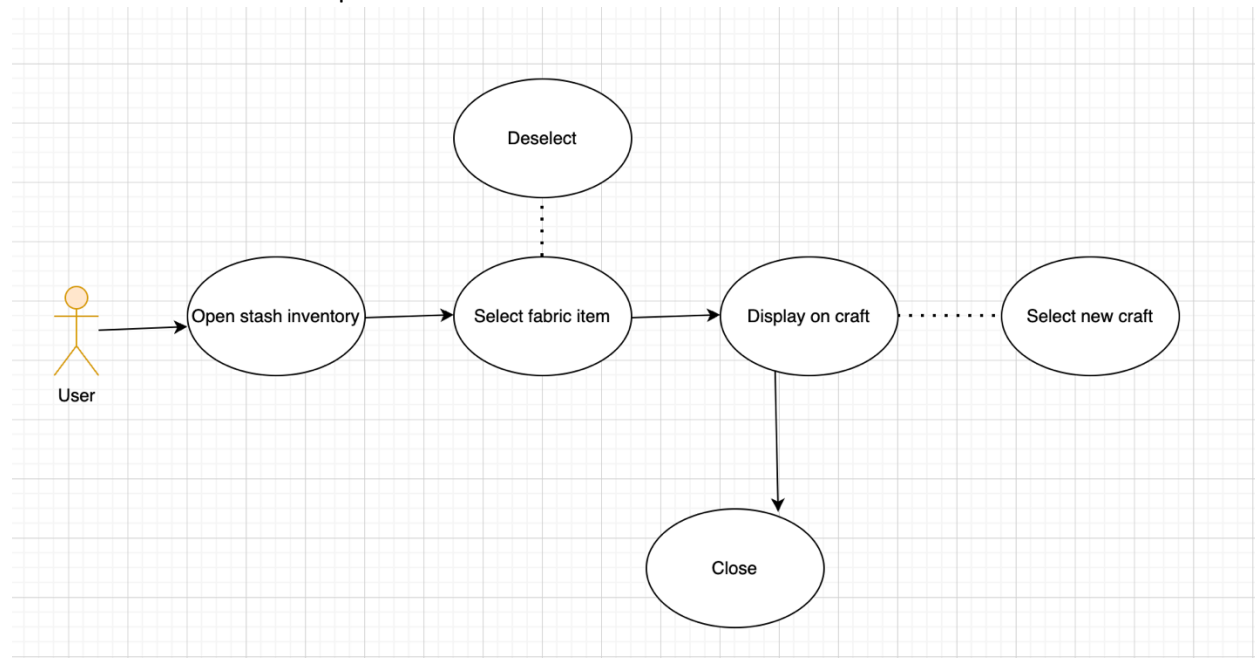
The calendar feature allows users to view details about available classes and sign-up if they would like to attend the class. In this use case, the user interacts with a web app's calendar function to view and register for available classes. The process begins when the user logs into the system and navigates to the calendar, where class schedules are displayed. The user selects a class from the available options and is shown detailed information about the class, such as date, time, and instructor. By clicking the "Sign Up" button, the user confirms their participation in the class. The system then registers the user and sends a confirmation, completing the sign-up process. If there is no space in the class, the user is shown a "Join Waitlist" button. The user can confirm their waitlist spot by clicking on this button. On the sponsor side of this feature, the class roster is displayed for each class with current sign ups. The system will add users to the class automatically after they sign up.

Case 2: User adds fabric to stash inventory

Use Cases	User adds fabric to stash inventory
Actors	Primary: User – Red
Preconditions	Must be logged in to web app Must have access to camera or device
Description	Be able to add a picture of fabric available to stash
Basic Flow	1. The User logs into web app 2. The user opens their stash inventory 3. The user is prompted on which way they want to upload their fabric item image 4. The user can either select the image through the device, or take a photo of the fabric 5. The user uploads the fabric item to the web app 6. The user's fabric is added to the database 7. The user is prompted to name fabric item 8. The new item gets stored in a database 9. The database uploads the fabric image to the stash inventory
Alternative Flow	1. The user can delete an item
Postconditions	The user successfully posted fabric item to their stash inventory The item is added to the database

	The user's stash inventory is refreshed and updated
Exceptions	The device has a network error The uploaded image is the wrong type The image and item details couldn't be uploaded to the Database

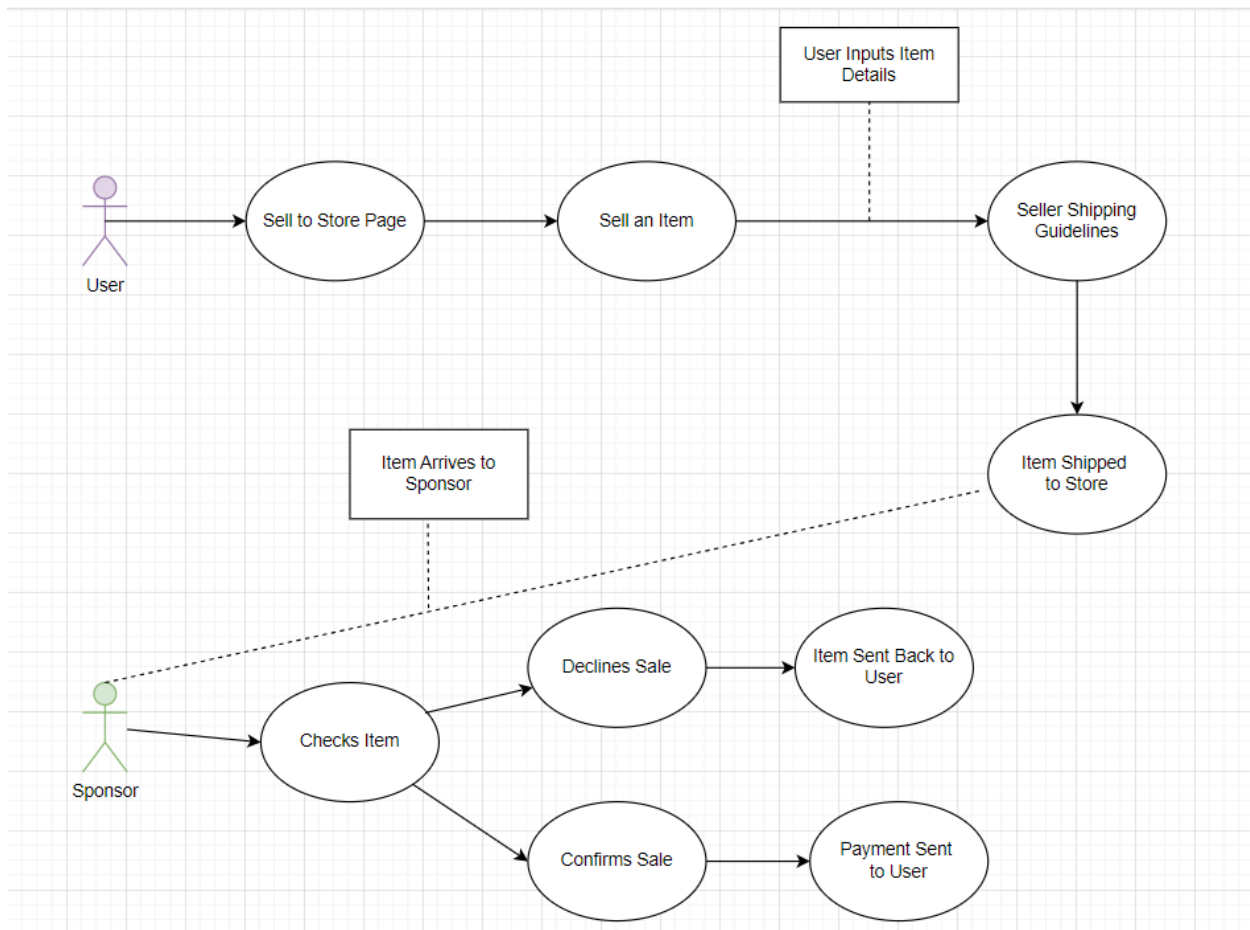
Our project scope includes the ability to upload an image of a fabric and create an item in the stash inventory. The user shall be able to customize the item with the image with names, and descriptions of the fabric. Our scope explains how customers often purchase fabrics they already own which leads to repetition and fabrics being tossed aside and thrown away. By allowing customers to add to their fabric stash it creates an online catalog for them to view, and update. While updating the customer's fabric stash inventory the user has the ability to either delete an item or continue selecting the option to add an item to the stash. When the add to stash option is clicked, the user will be prompted to either add an image by selecting from device or take a picture of the fabric. Once the image is uploaded, the user's image will be added to the database for storage, and the item will go through a creation pop-up that allows the user to name, categorize, and describe the fabric. Once done with processing, the user's item will be added to the stash inventory.

Case 3: User wants to view potential craft

Use Cases	User adds fabric to stash inventory
Actors	Primary: User – Orange
Preconditions	Must be logged in Must have fabric items in stash
Description	The user will be able to view what fabric items would look like on a craft by communicating with the database that holds a list of crafts.
Basic Flow	1. The User Opens Stash Inventory 2. The User Selects a Fabric Item 3. The System Prompts User to Display on craft 4. The User has the option to display fabric item on another craft
Alternative Flow	1. The User can deselect the item 2. The User can select a new craft option to view the fabric on a new craft 3. The user can close the display on craft menu
Postconditions	The user can view what their fabric would look like on a craft
Exceptions	The device has a network Error No crafts in database

Our scope outlines the ability to prompt the user to view what their fabrics would look like on crafts. The sponsor wanted to promote creativity to the community, and by allowing the user to view what potential crafts they can create, it avoids creativity blocks. This use case allows the user to navigate through the website to the stash inventory tab, and once they've uploaded a few items, or already have items in their stash, they can experiment with potential crafts by communicating with the database to have an archived list of what they can create.

Case 4: User Sells Item to the Store



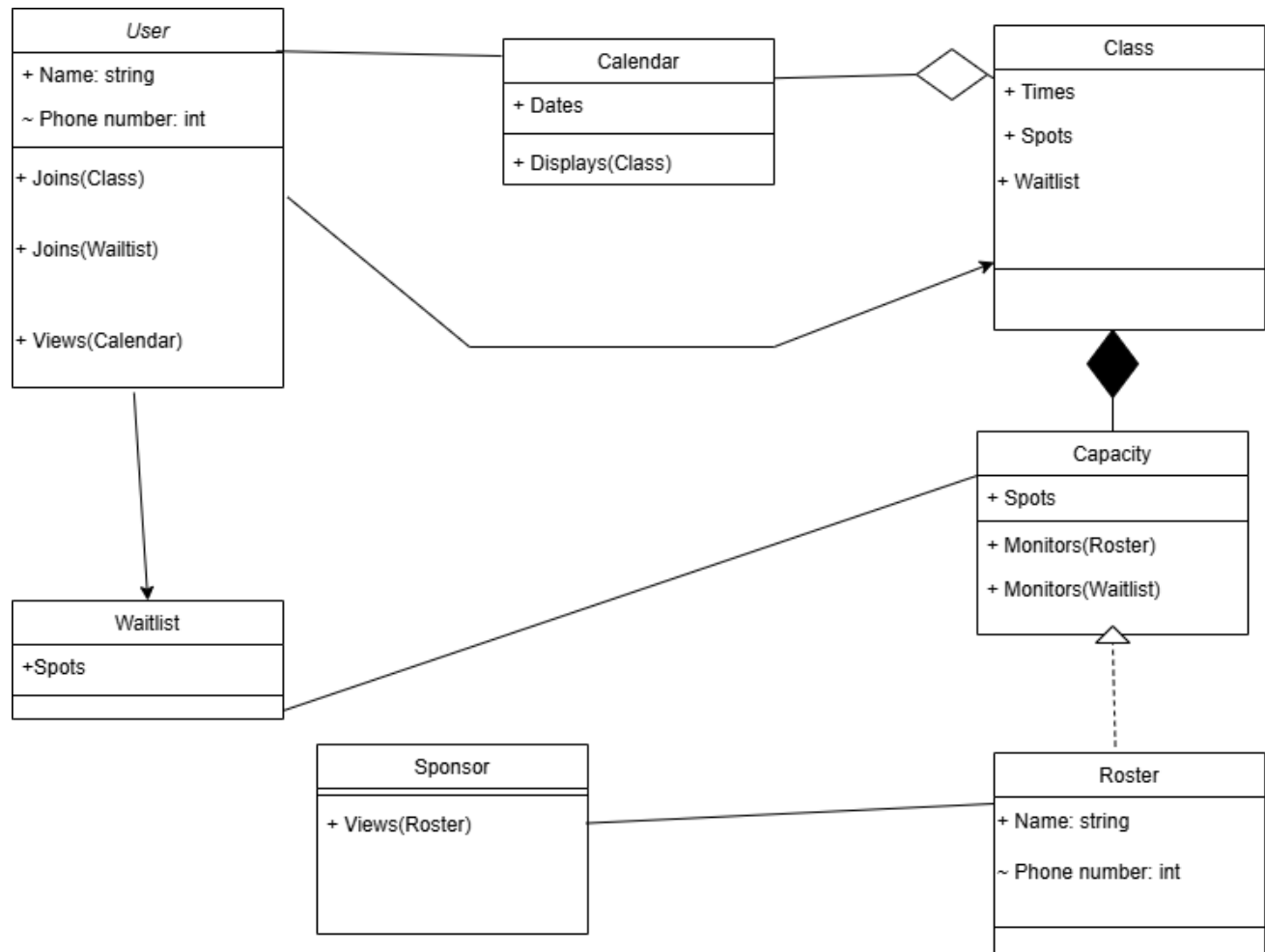
Use Case	User Sells Item to the Store
Actors	Primary Actor: User (Purple) Supporting Actor: Sponsor (Green)
Preconditions	The user must be logged into the app. The user must have access to the sell to store page.
Description	The user consigns an item they no longer need to the store through the app.
Basic Flow	1. The user logs into the web app. 2. The user selects the "Consign Item" option. 3. The system displays a page with an "Add Item to Consign" button. 4. The user clicks "Add Item to Consign". 5. The system displays various fields with different prompts to record item details. 6. The user fills out the fields.

	7. The system shows a “Confirm Consignment” button. 8. The user clicks “Confirm Sale”. 9. The system gives the user shipping instructions. 10. The user ships the item to the store. 11. The sponsor receives the item and confirms that the details match. 12. The sponsor puts the item up for sale in the store. 13. User gets paid consignment
Alternative Flows	1. The sponsor receives the item and declines the sale since the details do not match. 2. The sponsor sends the item back to the user.
Postconditions	The item is consigned to the store. The store pays the user. The item is added to the store’s inventory.
Exceptions	Network Failure: If the connection is lost during registration, the system notifies the user to try again later.

The consign to store feature allows users to consign an item that they no longer need to the store. In this use case, the user interacts with the app's consign-to-store function to sell the item. The process begins when the user logs into the system and navigates to the consign-to-store page, where a button is displayed. The user clicks the button and sees input fields for different details about the item. The user inputs the details and clicks “Confirm Consignment”. The system then gives the user shipping instructions for the item. The user ships the item to the store and once it arrives the sponsor will verify that the item matches the seller's description. The item will be verified, and the sponsor will put the item up for sale in the store. And once the item sells, payment will be sent to the user.

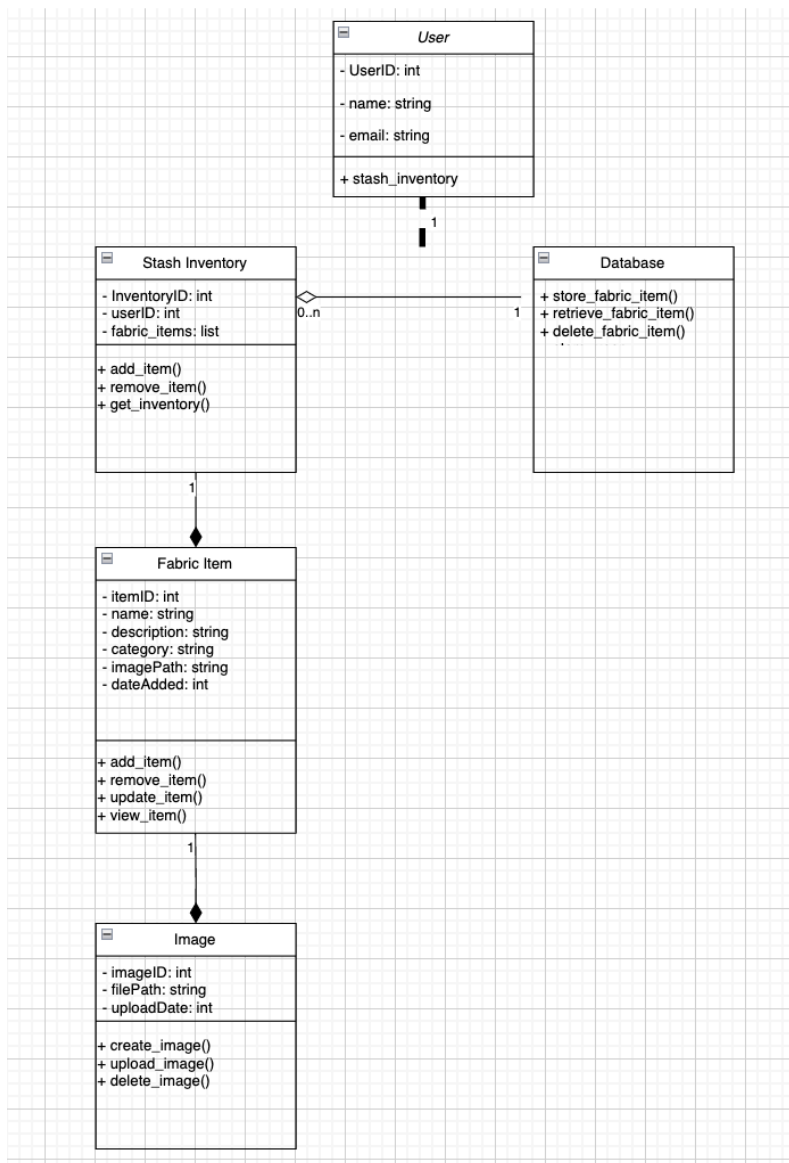
Logical View

Class Diagram for Case 1: User Signs Up for a Class via Calendar



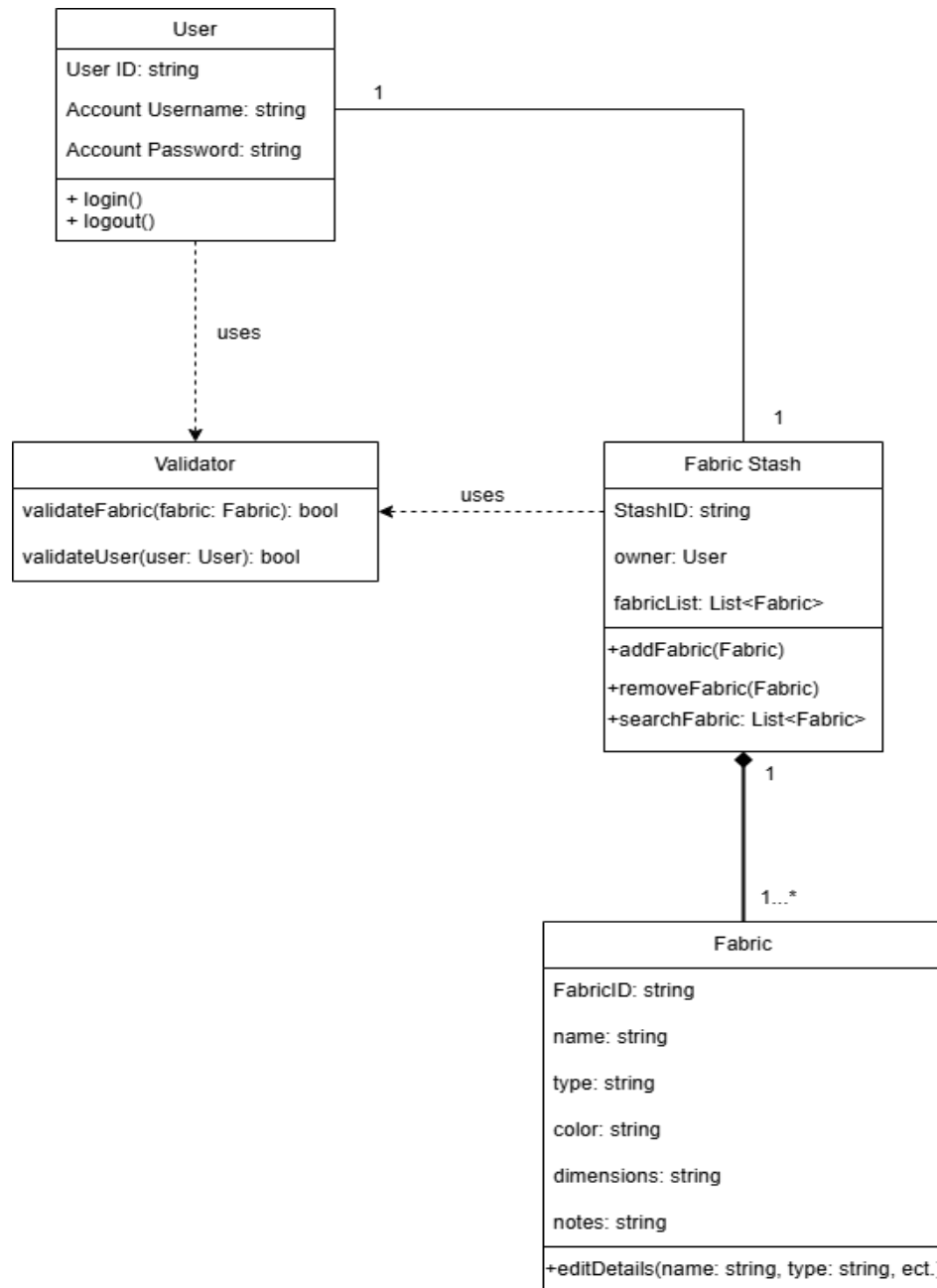
This class diagram represents use case 1: User Signs Up for a Class Via Calendar. The user's attributes are name and phone number. The user has a relationship with the calendar, class, and waitlist. The Calendar's attributes are dates and it has a holds-a relationship with the class. The calendar still exists with or without classes. The classes attributes are times, spots, and waitlist. There is a strong has-a relationship between the class and capacity since capacity cannot exist without a class. Capacity has spots as an attribute and has relationships with the roster and waitlist. The roster uses the capacity to know if it is full or not. The waitlists attribute is spots. Lastly, the roster has a relationship with the sponsor which is how the sponsor knows who will be attending the class.

Class diagram for case 2: User adds fabric item to stash inventory



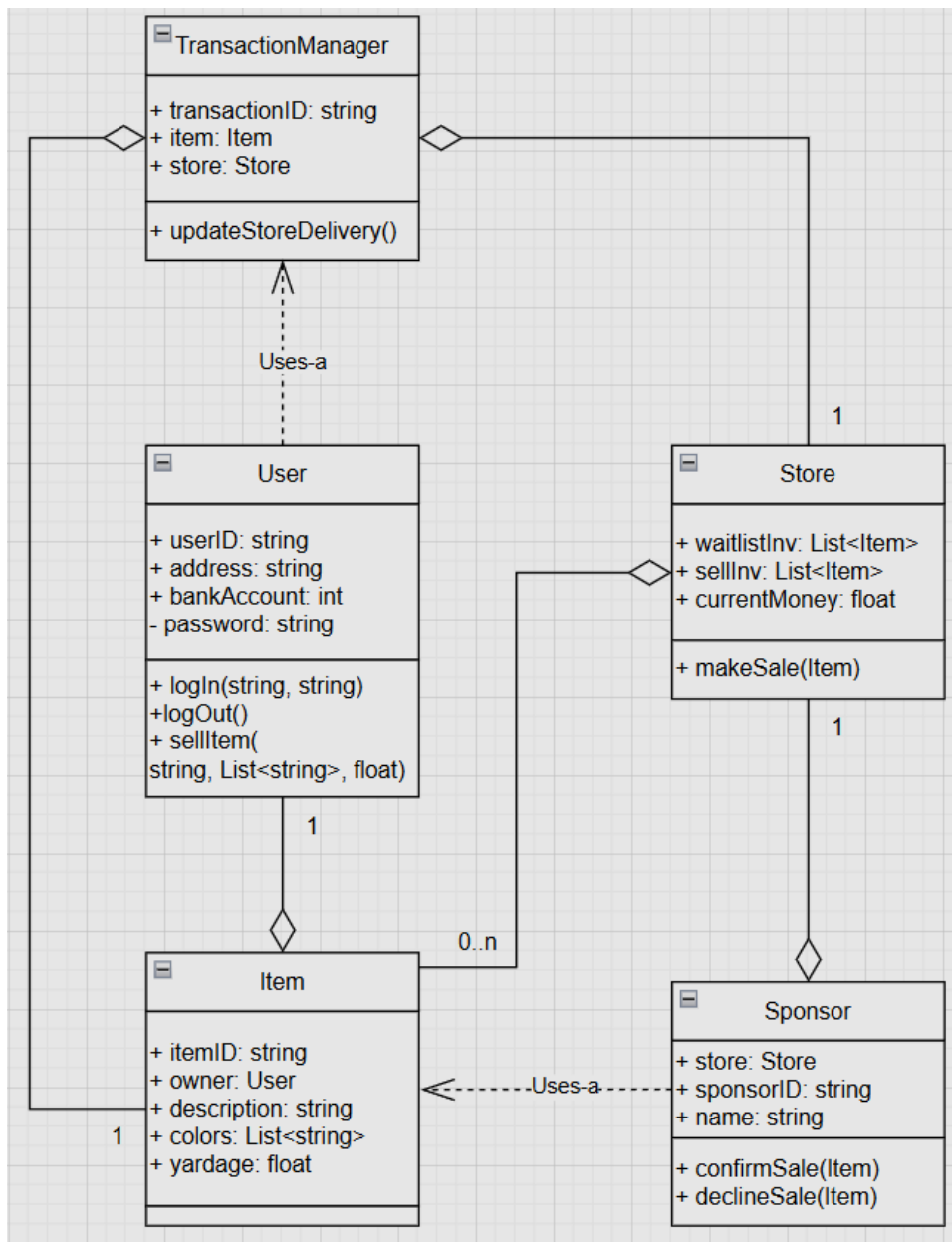
This class diagram represents the use case #2 where a user adds a fabric item to the stash inventory. The user will have a unique ID, a name associated with them, and an email. The user will have access to the stash inventory which has multiple attributes such as inventoryID, userID, fabric_items which will be a list of items. The stash inventory has methods to manipulate the inventory and add item objects to itself. The database will have a weak relationship with the inventory just to store its inventory. It doesn't own the inventory. The user has a strong relationship with the inventory. The inventory has a strong relationship with the fabric item which has the attributes of itemID, name, description, category, imagePath, and dateAdded with the methods add, remove, update, and view items. The fabric item class has a strong relationship with the image class. The image class has the attributes imageID, filePath, and uploadDate with the methods create, upload, delete image. It's important to note the relationships between the classes, and how the inventory gets updated to the database.

Class Diagram for Case 3: User adds stash to fabric inventory



This data diagram represents use case 3: The user's attributes are userID, account username, and account password. The user has a 1 to 1 relationship with their Fabric stash and has a uses relationship with the validator to access the fabric stash. The Fabric Stash has a stashID, an owner (User), and holds a list of fabrics and can be used to add, remove, and search fabrics from the list. The Fabric stash uses a "has-a" relationship with Fabric is has a one-to-many relationship as the fabric stash can contain many types of fabric, and a uses relationship with the validator when trying to add a new fabric. The fabric will hold many attributes which are used to define it such as name, type, color, dimensions, etc.

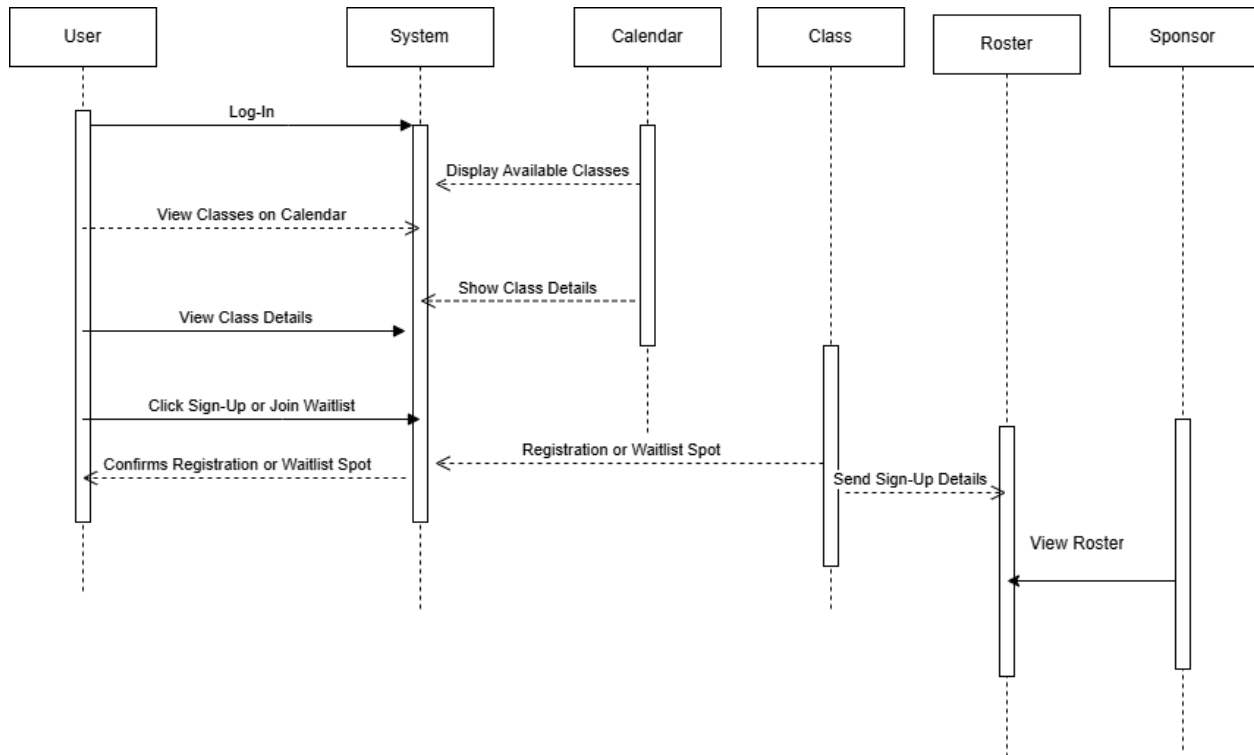
Class Diagram for Case 4: User Sells Item to the Store



This class diagram represents the use case of a user sending fabric to the store for consignment. Firstly, a User has login credentials (userID, password) along with a bankAccount and an address whether the User needs to get paid for the transaction or get the Item returned. A User can login and logout as well as sellItem which they include all the member variables necessary to create a new Item and a TransactionManager to manage the process. Item is created with the details provided by User along with a reference to the User that created it. The TransactionManager holds a reference to the Item and the Store so that the Item has a pointer until it arrives at the store, then it updateStoreDelivery() with a reference of the Item. The store holds two lists of Items, one that needs to be checked by a sponsor, and a list of actively selling items. It can makeSale(Item) from the sellInv list. Sponsor has a reference to Store so that it can access and confirmSale or declineSale Items from the waitlistInv.

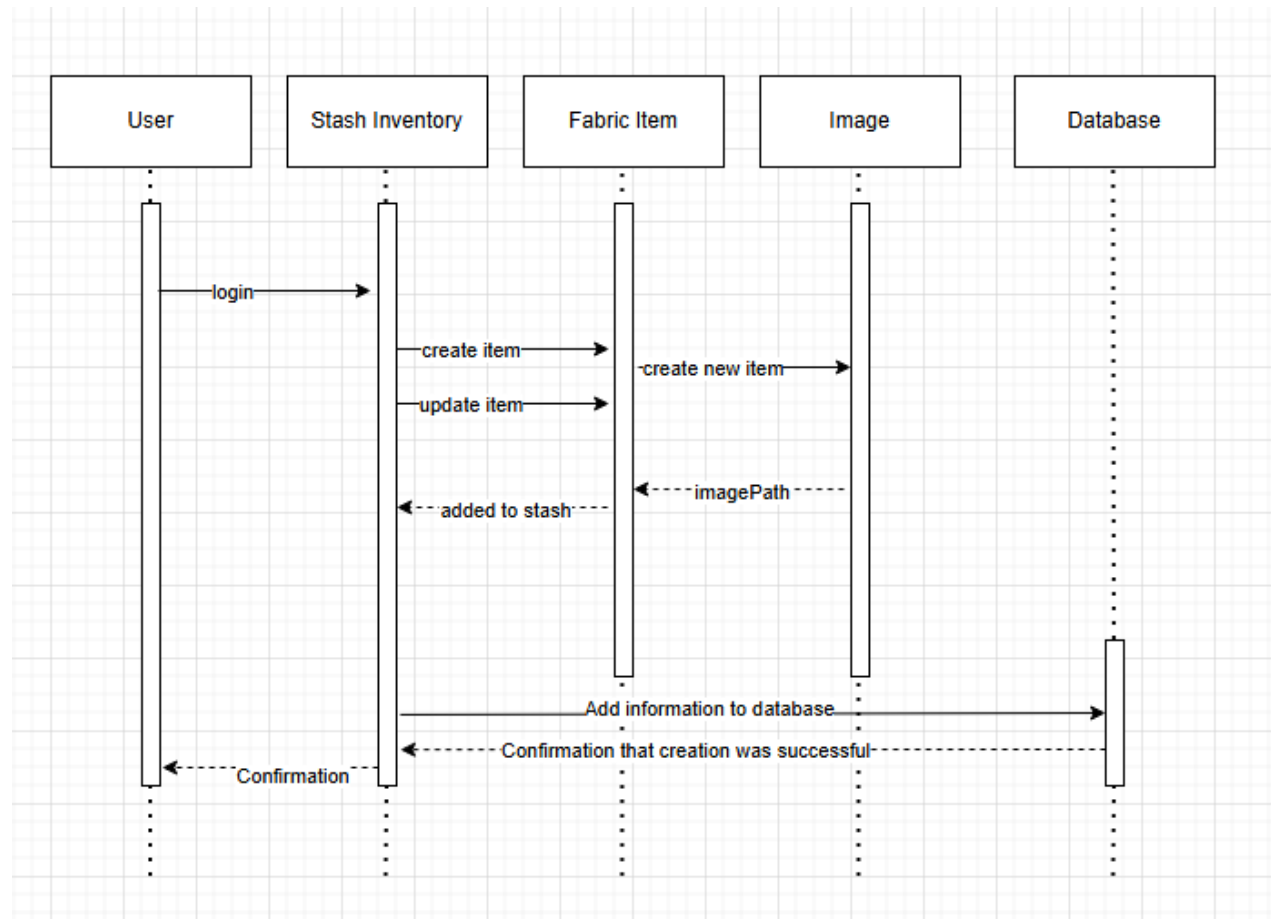
Process Diagrams

Sequence Diagram for Case 1: User Signs Up for a Class via Calendar



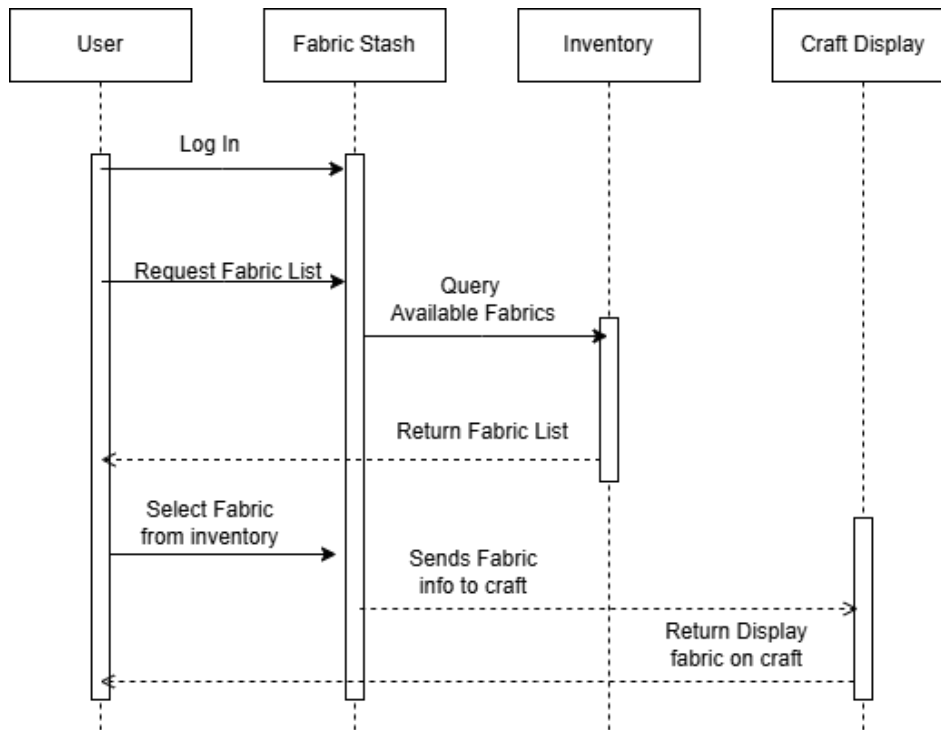
This sequence diagram shows the process of Use Case 1: User Signs Up for a Class via Calendar. It has interactions between the User, System, Calendar, Class, Roster, and Sponsor. The process begins with the user logging into the system. After logging in, they view classes on the calendar. The system retrieves and displays available classes through the calendar interface. The user then selects a specific class to view its details. The system retrieves and displays details of the class through the calendar interface. The user then clicks either the “Sign-Up” or “Join Waitlist” option, depending on class availability. The system then confirms the user's registration or their position on the waitlist. The system sends the user's sign-up details to the class roster. The sponsor has the ability to view the class roster, which reflects all registered users.

Sequence Diagram for case 2: User adds fabric item to stash inventory



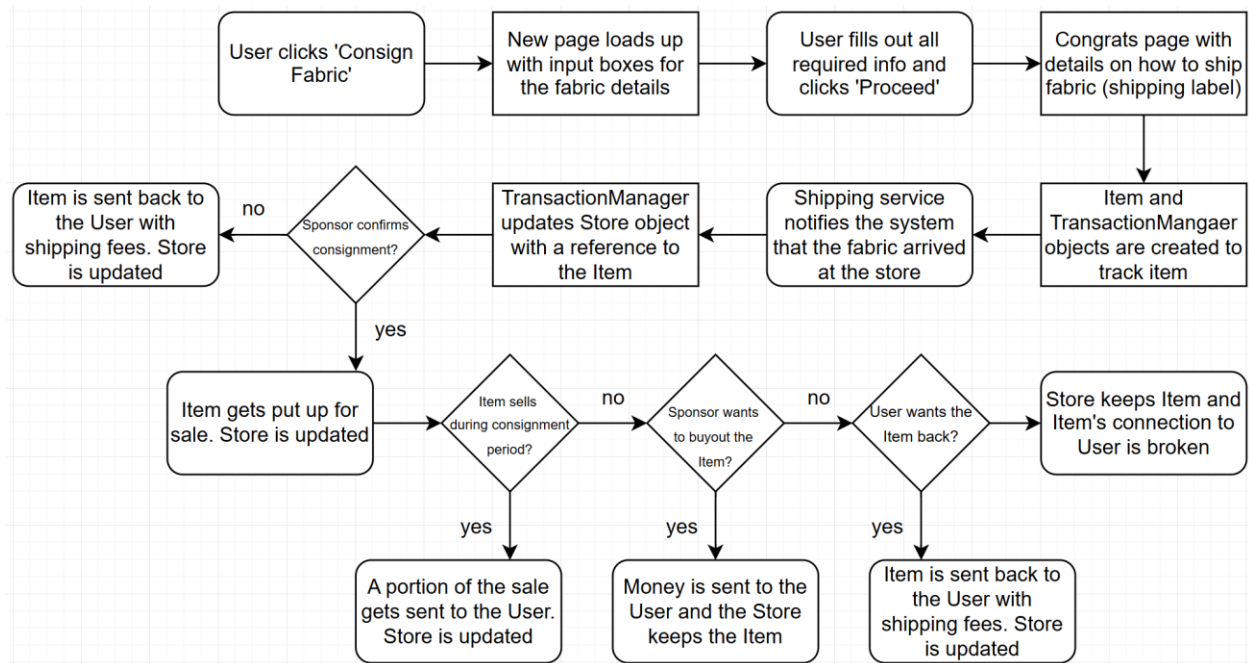
This sequence diagram shows the process of a user adding a fabric item to their stash inventory and sending that information to be stored in a database. It has interactions between the user, stash inventory, fabric item, image, and the database. The user initiates the action by calling the add Item() method on the stash inventory. When they click on the add item selection, a pop up will open to prompt the user on how they want to upload the image. The image path and image will be sent to the fabric item, which will send the item details to the inventory, and will end its activity. The inventory will send the information to the database for storage and will get confirmation from the database that the item was successfully created, then sent to the user.

Sequence Diagram for case 3: User views potential craft



This sequence diagram shows the process of Use Case 3: User views potential craft. There are interactions between the User, Fabric Stash, Inventory, and Craft Display. The user begins by logging into the fabric stash. Once logged in the user requests to see their fabric list, the fabric stash then gets this information from the Inventory. After receiving the list, the user selects the fabric they want to view on a craft, the fabric stash sends the fabric info to the display which is then returned to the user.

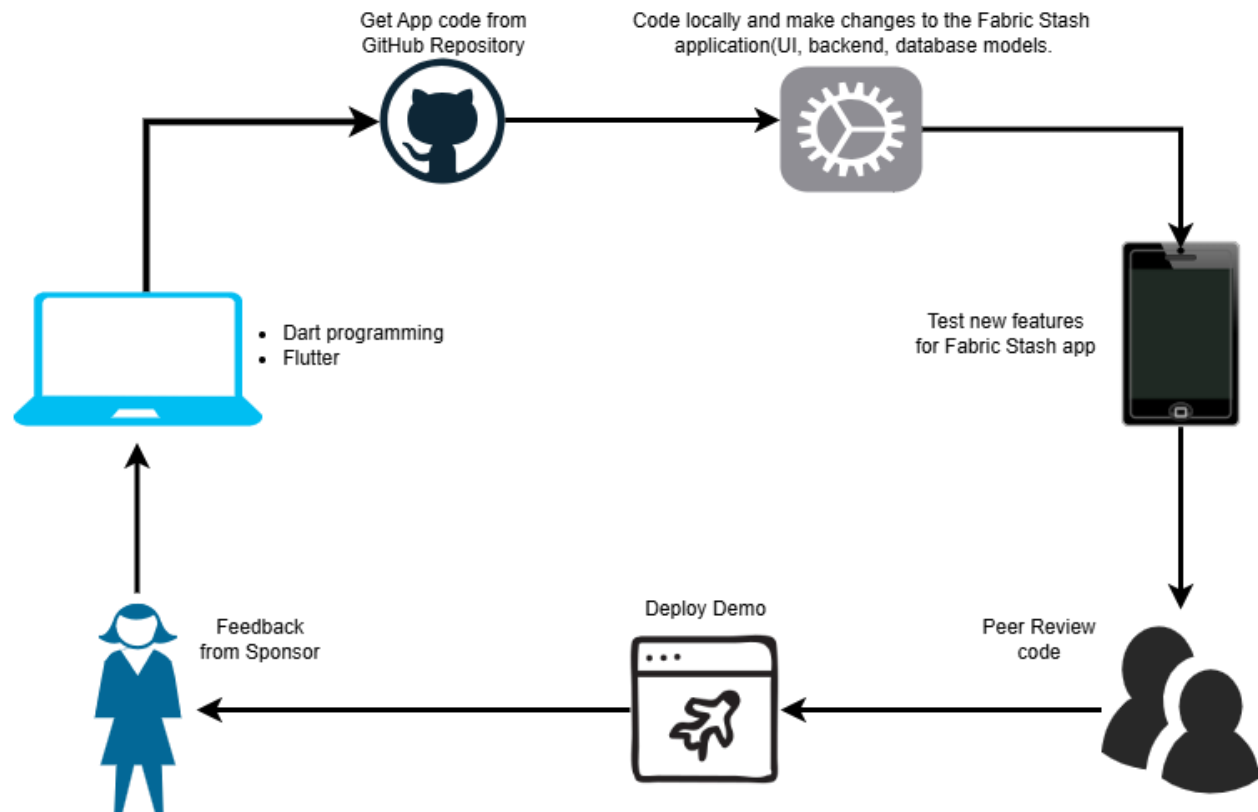
Activity Diagram for Case 4: User Sells Item to the Store



This activity diagram walks through all of the actions and responding system updates that occur when a User is consigning an Item to the Store. Firstly, the User decides to consign some fabric they have, so they click the 'Consign Item' button in the app. Clicking this button takes the user to another page where they must fill out all the required fields describing the fabric. Once all the required fields are filled, the User clicks 'Proceed' which takes them to a new page. This page pops up with a congratulations message and information on how the User can ship the fabric to the store such as the shipping label, nearest UPS store, etc. With the information provided by the User, an Item and TransactionManager object is created to track the Item during this process. Eventually, the shipping service will notify the system that the fabric has arrived at the store. This will cause the TransactionManager object to update the Store object's waitlistInv with the Item that arrived. When the Sponsor checks the Item, they will either confirm or decline that the fabric is sellable. If the fabric is confirmed, the fabric will be put up for sale for a consignment period. If the item gets sold within the consignment period, the User gets sent a portion of the sale value. If the consignment period is over and the fabric is not sold, the store will either offer a buyout (buy the fabric from the user), or give the User the option to donate the fabric to the store or get the fabric sent back to them. However, if the transaction is denied, the Item will get shipped back to the User along with a bill to cover the cost of shipping. The Item will also be removed from the Store's waitlistInv.

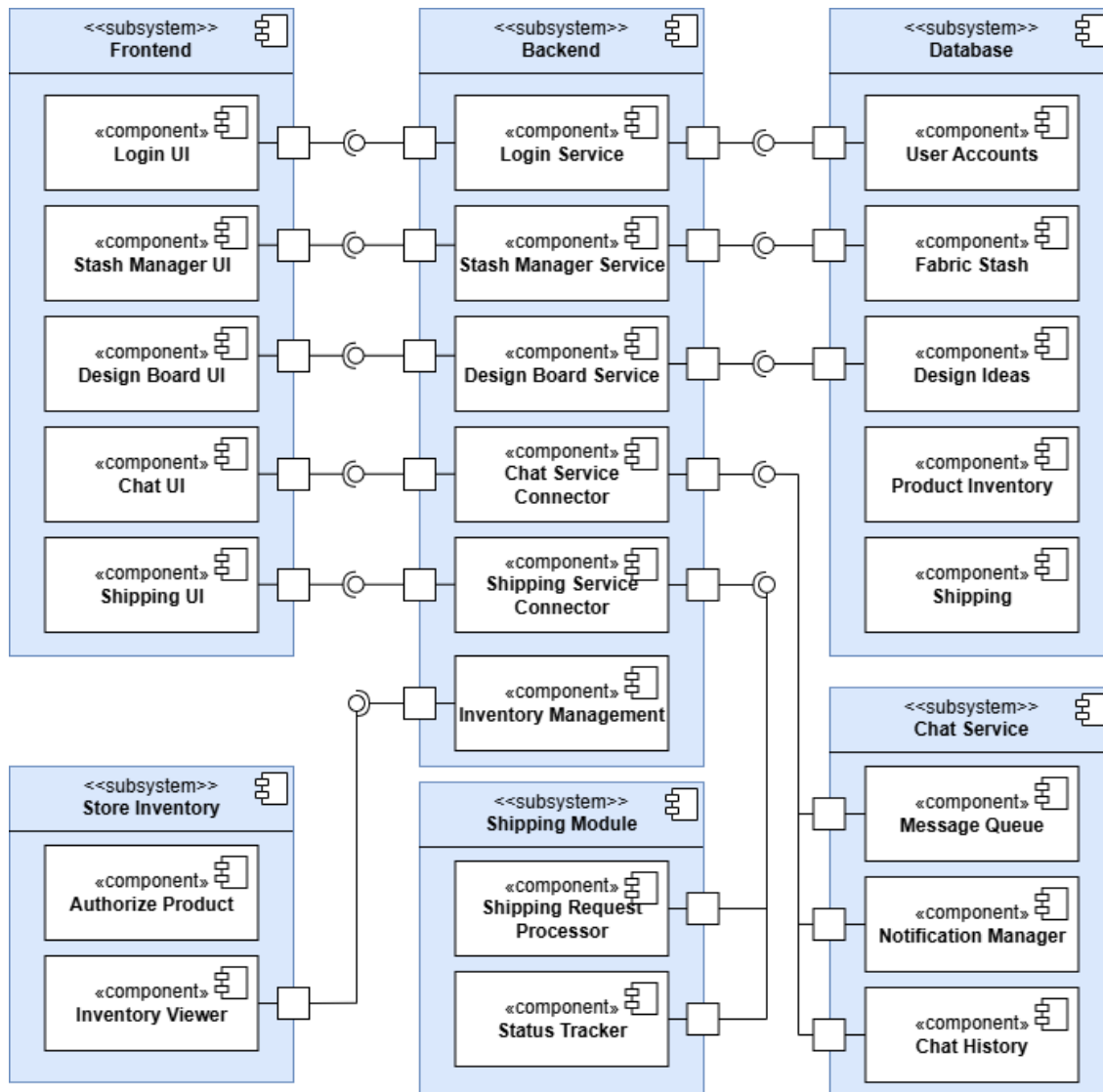
Development Diagrams

Developer Dev Ops Deployment Diagram:



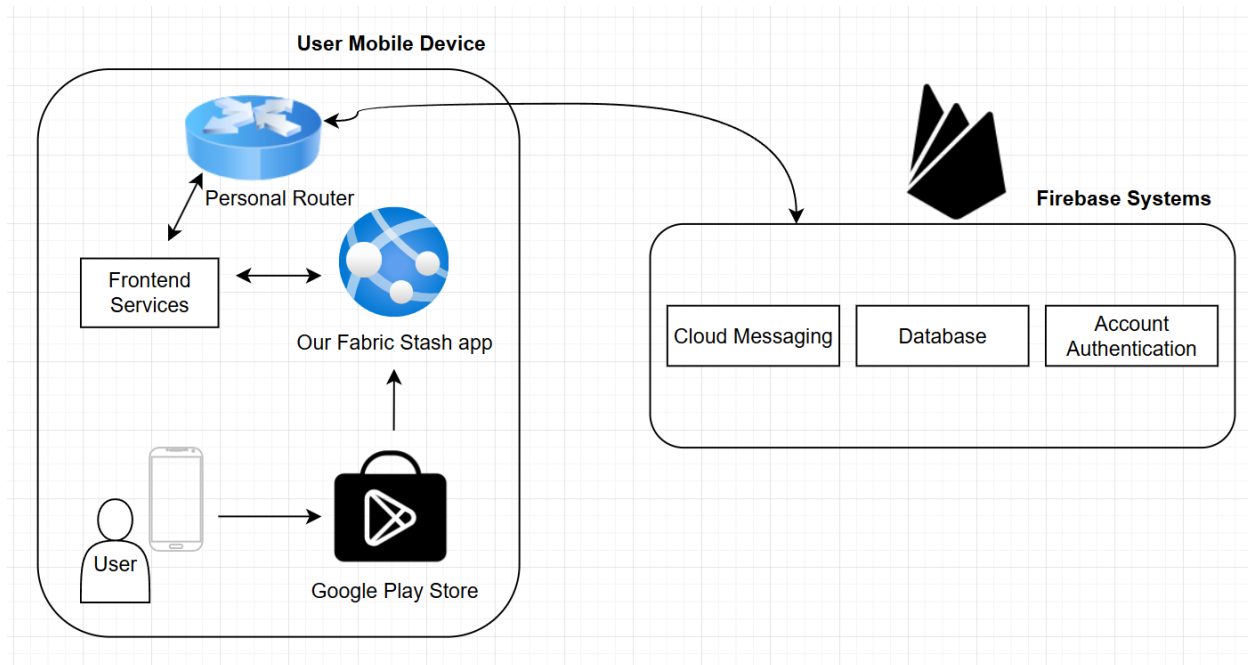
The Developer DevOps Deployment Diagram shows the step-by-step process of how we will be further developing the Fabric stash app and features. We start the process by setting up development apps and languages (for our project this being Dart and Flutter), and then we will proceed to access the preestablished code we already have in our GitHub repository, we can begin the update/development phase where the bulk of the programming is done for updates to the app or development of new features. After the development phase, we move onto the testing phase in which there will be both manual and automated testing and will also be peer reviewed. Once testing is complete and the updated app or new feature is working properly, we move onto the deployment of a demo which will allow us to showcase the new updates or features to our sponsor and get feedback before starting the cycle over again.

UML Component Diagram:



This UML component diagram explains how we will structure our code and subsystems to support a functional app with the specifications we need. There are six main subsystems: Frontend, Backend, Databases, Store Inventory, Shipping Module, and Chat Service. The main flow of data is from Databases (tables storing raw data), to Backend (any required computation will be complete), and finally to Frontend (where the data will be shown on screen to the user). However, some of the data will need to be manipulated by other subsystems which breaks down large tasks into more conceivable ideas. These subsystems are: Store Inventory (which manages the inventory of the store in person), Shipping Module (tracks and manages shipments to the store and back to the user), and Chat Service (manages the chats between users). These subsystems make up the logical and physical components of our UML component diagram.

Physical Diagram



This physical diagram illustrates the physical deployed state and the data flows between the app's services. This physical diagram represents the deployment to android devices as it will be the quickest platform to start tests on. The first-time users want to use the app, they must download it from the Google Play Store. Once downloaded, the user will be able to open up the app from their phone and access the frontend of the app (UI). When the app starts up, they will have to log in or create an account, this will be done via Firebase Services. We chose to go with Google Firebase Services because the pricing is free for development and low-traffic applications, which we imagine this app will be. Also, it is a great platform that has many services we can pick from to help us design our app better without having to code everything from scratch. The user must be connected to the internet so that their phone can route them to Firebase Account Authentication to authenticate the login process. Once users have logged in, they will make backend requests from the app's frontend. So, the data will flow from the frontend service to the Firebase Gateway which will fetch the backend functionality from code on Firebase servers. This may be a query to the database (we chose MongoDB for the pricing) or to access the notification hub. If a notification is to be pushed to the user, the notification hub will interact with Firebase Cloud Messaging (FCM). FCM is the notification service that android phones use. The FCM service will then flow directly to the user's router and onto their phones via the app. If the frontend request was a database query or just to achieve some backend service such as the messaging service, data will flow from the Firebase servers, to the Firebase Gateway, to the user's personal router, to the app's frontend which the user can access from the app.

Version Control

Date	Revision	Author	Change
10/24/24	1.1	Everyone	Initial Creation
10/29/24	1.2	Everyone	Addressing Feedback for Resubmit
10/31	1.3	Everyone	Logical View Diagrams
11/5	1.4	Everyone	Process Diagrams
11/12	1.5	Everyone	Updates and Development Diagrams
11/19	1.6	Everyone	Physical Diagram
11/27	1.7	Everyone	Final Signoff Tweaks
2/11/25	1.8	Rithikaa & Oscar	WQ Updates